



分布与并行数据库

4 分布式查询处理与优化

主讲教师：鄂金龙
中国人民大学信息学院





主要内容

- 查询处理基础
- 查询处理器
- 查询分解
- 数据局部化
- 片段查询的优化
- 大数据库的查询处理及优化

查询处理基础

□ 优化目标

优化就是寻找执行代价（费用和时间）最小的查询执行策略，使系统执行效率降到最低。**优化的目标**就是指局部执行代价和网络传输代价的和最小。

- **局部执行代价**：主要指输入/输出次数（I/O代价）及CPU处理代价。
- **网络传输代价**：主要指传输启动代价和数据传输代价。

查询处理基础

□ 执行策略

我们以一个例子来说明选择何种执行策略。

例：求选修2号课程的学生姓名

■ SQL表示：

```
select Sname  
from Students, SC  
where Students.Sno=SC.Sno and Cno='2';
```

■ 关系代数表示：

$Q1 = \pi_{\text{sname}}(\sigma_{\text{Students.Sno}=\text{SC.Sno and Cno}='2'}(\text{Students} \times \text{SC}))$

$Q2 = \pi_{\text{sname}}(\sigma_{\text{Cno}='2'}(\text{Students} \bowtie \text{SC}))$

$Q3 = \pi_{\text{sname}}(\text{Students} \bowtie \sigma_{\text{Cno}='2'}(\text{SC}))$

$$Q1 = \pi_{sname}(\sigma_{Students.Sno=SC.Sno \text{ and } Cno='2'}(Students \times SC))$$

查询处理基础

□ 代价计算

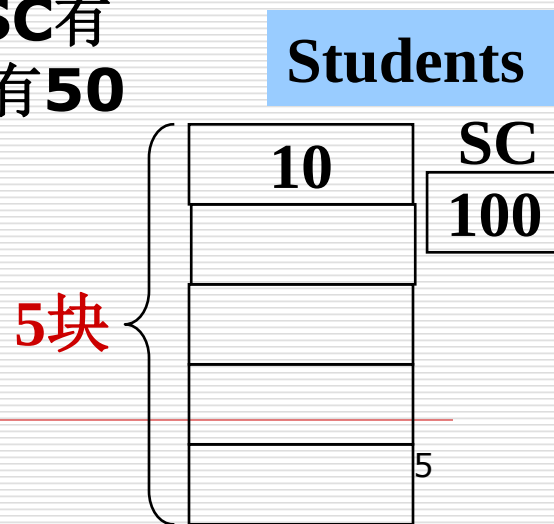
■ Q1代价计算(仅考虑I/O代价)

✓ 计算广义笛卡尔积代价

-**假定**: 在内存中, 存放**5块Students**元组和一块**SC**元组, 一块可以装**10个Students**元组或**100个SC**元组.

-**假定**: **Students**有**1000**个元组,**SC**有**10000**个元组, 其中选**2号课程**的有**50**个元组

-数据只有读到内存才能进行连接



$Q1 = \pi_{\text{sname}}(\sigma_{\text{Students.Sno}=\text{SC.Sno and Cno}='2'}(\text{Students} \times \text{SC}))$

查询处理基础

✓ 通过读取块数计算I/O代价

读取块数计算方法:

Students 1000个元组

SC 10000个元组

• 读取总块数:

SC块数

$$\frac{1000}{10} + \frac{1000}{10 \times 5} \times \frac{10000}{100} = 100 + 20 \times 100 = 2100$$

• 若每秒读写**20**块, 则花费:

$$\frac{2100}{20} = 105s$$

Students

10

SC

100

5块

Student
读入内存
的次数

Student
块数

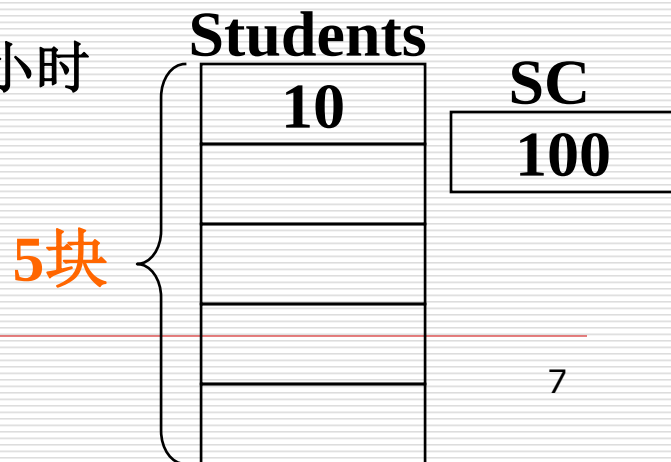


$$Q1 = \pi_{\text{sname}}(\sigma_{\text{Students.Sno}=\text{SC.Sno and Cno}='2'}(\text{Students} \times \text{SC}))$$

查询处理基础

- ✓ **条件**: Students 1000个元组, SC 10000个元组
 - 笛卡尔集计算后的元组个数为: $10^3 \times 10^4 = 10^7$
 - 连接后的中间结果内存放不下, 需暂时写到外存
 - 若每块可装**10**个, 完成笛卡尔集后的元组, 则写这些元组需: $(10^7 / 10) / 20 = 5 \times 10^4 \text{s}$
 - 选择操作**: 读回需 $5 \times 10^4 \text{s}$, 假设选择后剩50个元组, 均可放在内存
 - 投影操作: 0s
 - 查询共花费: **每秒读20块**

$$105 + 2 \times 5 \times 10^4 \approx 10^5 \text{ s} \approx 28 \text{ 小时}$$



$$Q2 = \pi_{sname}(\sigma_{Cno='2'}(Students \bowtie SC))$$

查询优化基础

■ Q2代价计算(仅考虑I/O代价)

✓ 计算自然连接代价

- 也要把数据读到内存进行连接, 但连接结果比笛卡尔积要小得多

- 读取块数依然为: $\frac{1000}{10} + \frac{1000}{10 \times 5} \times \frac{10000}{100} = 100 + 20 \times 100 = 2100$

花费为 **2100/20=105s**

- 假设连接结果大小为 **10⁴** 个元组, 写到外存需:

$$(10^4 / 10) / 20 = 50s$$

每秒读20块

$$Q2 = \pi_{sname}(\sigma_{Cno='2'}(Students \bowtie SC))$$

查询处理基础

- 读取自然连接结果，执行选择运算，需50s，选择结果均可放在内存
- 投影运算：0s
- 总花费为：105+50+50=205s $\approx$ **3.4分钟**

$$Q3 = \pi_{sname}(Students \bowtie \sigma_{Cno='2'}(SC))$$

■ Q3代价计算(仅考虑I/O代价)

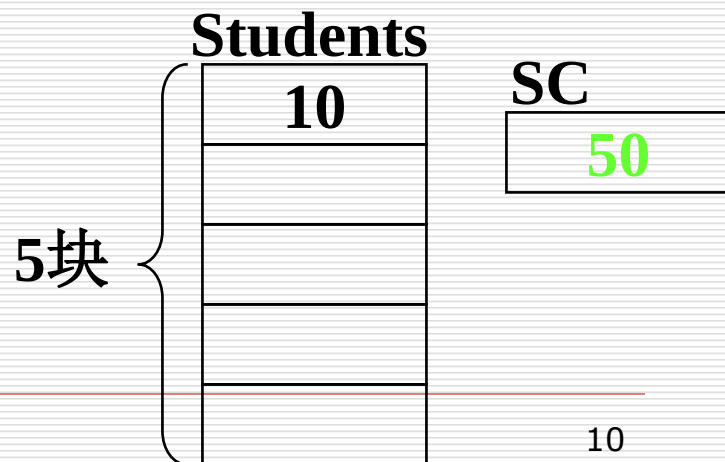
✓ 计算对SC做选择运算的代价

- 需读SC到内存进行选择运算
- 读SC块数为：10000/100=100
- 花费为：100/20=5s
- 选择结果为50个SC元组，均可放在内存

$$Q3 = \pi_{sname} (Students \bowtie \sigma_{Cno='2'}(SC))$$

查询处理基础

- ✓ 计算和Students自然连接的代价
 - 需读Students到内存进行连接运算
 - 读Students块数为:
 $1000/10=100$
 - 花费为: $100/20=5s$
 - 连接结果为50个元组, 均可放在内存
 - 投影运算: 0s
- ✓ 总花费: $5+5=10s$



查询处理基础

上述三种策略看，虽然都能实现所要完成的功能，但三种实现方法所需的代价却相差很大。

在分布式系统中，对于多用户、多应用需求的复杂任务，采用不同的实现策略会相差更大，将直接影响整个系统整体性能。

因此，需要确定出一种执行代价最小的查询执行策略。

查询处理基础

□ 优化内容

优化内容体现如下几点：

- 执行运算的次序。
- 执行每种运算的方法。如上例，不同方法代价不同。
- 所访问的副本场地。如：选择就近的场地，节约传输代价。
- 执行运算的场地的选择，使总的传输代价或总代价最低。

综合考虑，确定出一种执行代价最小的查询执行策略。

查询处理基础

- 用户或应用看到的是全局关系组成的全局数据库，用户通过查询语言（通常用**SQL**语言）来表达**全局查询**。之后，由系统将其转换成**等价的关系表达式内部表示**，为描述关系的操作序列，提出一种称**查询树**的内部表示方法。
- **关系代数**
 - 一元运算
 - u**: σ （选择）/ π （投影）
 - 二元运算
 - b**: $\bowtie$ （联接）/ $\ltimes$ （笛卡儿积）/ $\cup$ （并）/ $\cap$ （交）/ $-$ （差）/ $\ltimes$ （半联接）

查询处理基础

□ 等价变换

重复律: $\mathbf{uR} \equiv \mathbf{uuR}$

交换律: $\mathbf{u_1u_2R} \equiv \mathbf{u_2u_1R}$

分配律: $\mathbf{u(RbS)} \equiv (\mathbf{uR})\mathbf{b(uS)}$

结合律: $\mathbf{Rb_1(Sb_2T)} \equiv (\mathbf{Rb_1S})\mathbf{b_2T}$

提取律: $(\mathbf{uR})\mathbf{b(uS)} \equiv \mathbf{u(RbS)}$

其中: $\mathbf{R}$ 、 $\mathbf{S}$ 、 $\mathbf{T}$ 为关系, $\mathbf{u_1}$ 、 $\mathbf{u_2}$ 、 $\mathbf{u}$ 为一元运算符, $\mathbf{b_1}$ 、 $\mathbf{b_2}$ 、 $\mathbf{b}$ 为二元运算符。

查询处理基础

□ 查询树

在查询树中，**叶子**表示关系，**中间节点**表示运算，前序遍历关系表示运算次序。

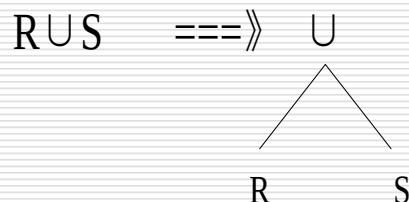
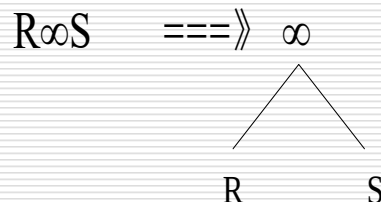
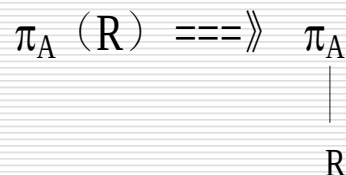
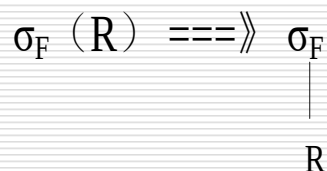
■ 定义：

ROOT=T

T=R / (T) / TbT / uT

u=σ_F / π_A

b=∞ / X / U / ∩ / - / ∝



查询处理基础

□ 举例：设有一供应关系数据库，有供应者和供应两关系，如下：

供应者：**SUPPLIER{SNO, SNAME, AREA}**

供应者编号

供应者姓名

供应者所属地域

供应：**SUPPLY{SNO, PNO, QTY}**

供应者编号

零件号

数量

查询要求：找出地域在“北方”供应**100**号零件的供应商的信息。

SQL查询语句：**SELECT SNO, SNAME
FROM SUPPLIER, SUPPLY
WHERE AREA="北方"AND
PNO=100 AND
SUPPLIER.SNO=SUPPLY.SNO**

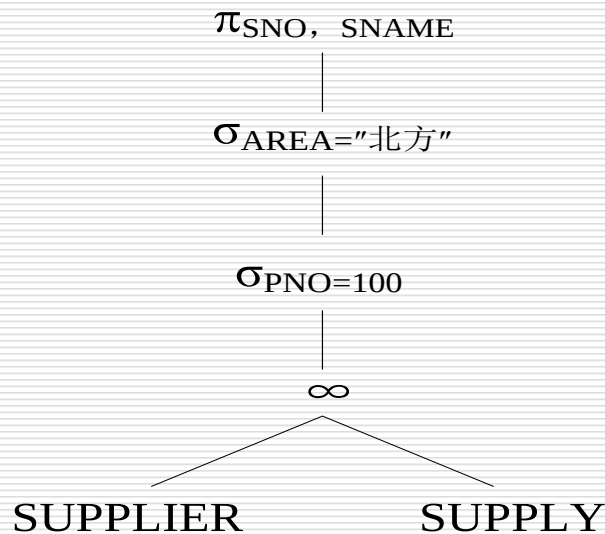
查询处理基础

□ 举例

等价的关系表达式:

Q1: $\Pi_{SNO, SNAME} (\sigma_{AREA="北方"} (\sigma_{PNO=100} (SUPPLIER \bowtie SUPPLY)))$

查询树:



查询处理器

- **影响查询处理效率的因素有：**网络传输代价（数据量和延迟等）、局部**I/O**代价及**CPU**处理代价等，但主要由**网络通信代价和局部I/O代价**来衡量。不同的分布式数据库系统可能对评估查询处理的**传输代价和I/O代价**的侧重不同。
- 为提高查询的效率，在查询处理过程中还要进行优化处理。
- **查询优化**就是确定出一种执行代价最小的查询执行策略或寻找相对较优的操作执行步骤。一般采用**多级优化**的策略。本章介绍全局查询的处理与优化。

查询处理器

□ 查询语言:

SQL—非过程化的关系数据库语言，是元组演算语言

■ 元组演算: $\{t | F(t)\}$

其中， t : 元组变量， $F(t)$: well-formed formula(wff),

✓ 例如: 查询所有 managers 的 no. 和 name。

$$\{t(ENO, ENAME) | t \in E \wedge t(TITLE) = "MANAGER"\}$$

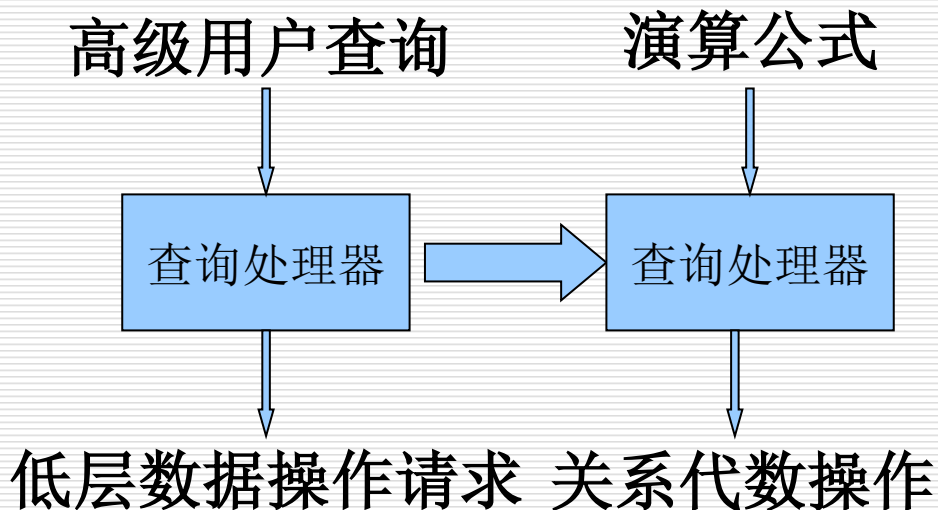
■ 域演算: $\{x_1, x_2, \dots, x_n | F(x_1, x_2, \dots, x_n)\}$

其中 x_1, x_2, x_n : 域变量, $F(x_1, x_2, \dots, x_n)$: wff

例如: $\{x, y | E(x, y, "manager")\}$

查询处理器

- 查询处理将查询转换为过程操作来访问数据。



- 分布查询处理器还需要处理查询分解（**query decomposition**）和数据局部化（**data localization**）

查询处理器

□ 查询处理问题

■ 集中查询处理器必须：

- ✓ 将演算查询转换为代数操作
- ✓ 选择最好的执行计划

■ 例如：

SELECT ENAME

FROM E,G

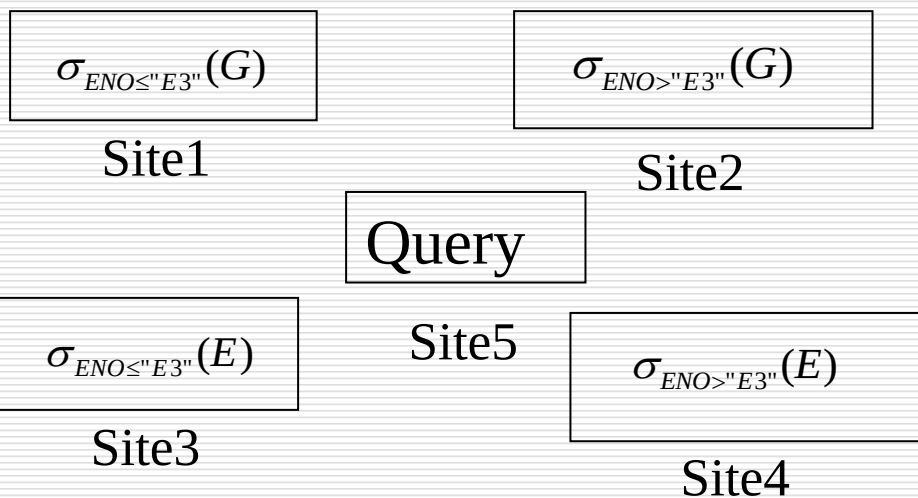
WHERE RESP = “Manager” and
E.ENO=G.ENO

查询处理器

- 关系代数 1: $\pi_{ENAME}(\sigma_{RESP="Manager" \wedge E.EMO=G.ENO}(E \times G))$
- 关系代数 2: $\pi_{ENAME}(E \bowtie_{ENO}(\sigma_{RESP="Manager"}(G)))$
- 可见: 执行计划 2 好
- 对于分布式数据库, 查询处理器必须考虑通信代价和选择最佳场地!
- 如上例: 若 **G** 和 **E** 是分布的.
- 简单计划是将所有片段传到查询场地并执行, 但通信代价太大。

查询处理器

□ 分布式查询举例：

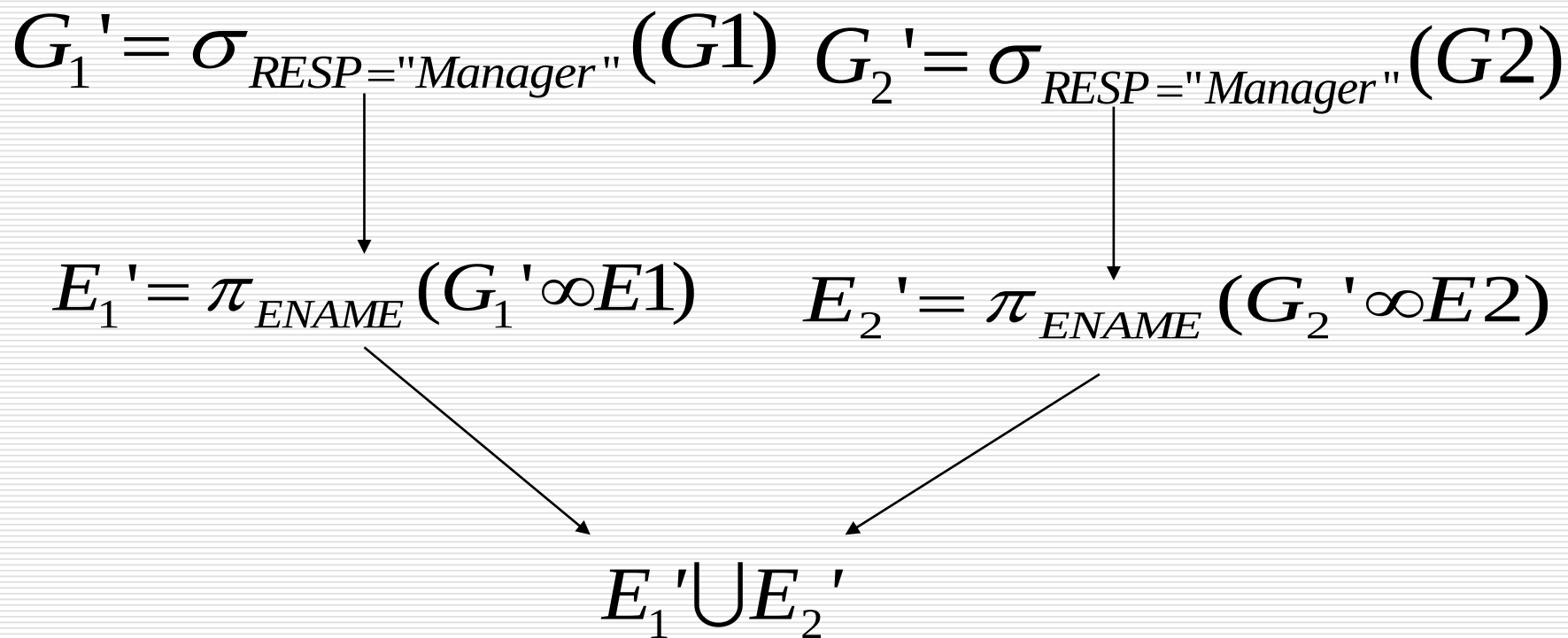


■ 查询：

$$\pi_{ENAME}(E_{\infty ENO}(\sigma_{REPS="Manager"}(G)))$$

查询处理器

■ 优化的查询



查询处理器

□ 查询处理的目标

■ 两方面：

转换（**transformation**）和优化（**optimization**）

■ 考虑的优化代价：

✓ **CPU time**

✓ **I/O time**

✓ **Communication time**

■ 在WAN内，侧重通信代价 在LAN内，三者同等重要

查询处理器

□ 优化类型

- 穷举法 (**Exhaustive search**) – 用于小规模解空间
- 启发式 (**Heuristics**) – σ , π 优先, 然后是半连接等, 用于大规模解空间

查询处理器

□ 优化时机

- **静态的(Static)** – 在编译时进行优化，基于统计信息进行穷举优化，优化一次，执行多次
- **动态的(Dynamic)** – 在执行中优化，准确性好，但每次执行都进行优化，代价高

查询优化器

□ 统计数据 (Statistics)

元组基数, 属性值分布, 关系大小等。信息提供给查询优化器并定期修改。

□ 决策场地 (Decision Site)

查询优化场地可以:

- 单场地: 集中的方法, 简单, 需要整个分布库的知识。
- 分布式: 涉及所有的场地协调确定调度, 只需本地知识, 需要协调代价。
- 混合型: 一个场地确定全局调度, 其他场地优化局部子查询。

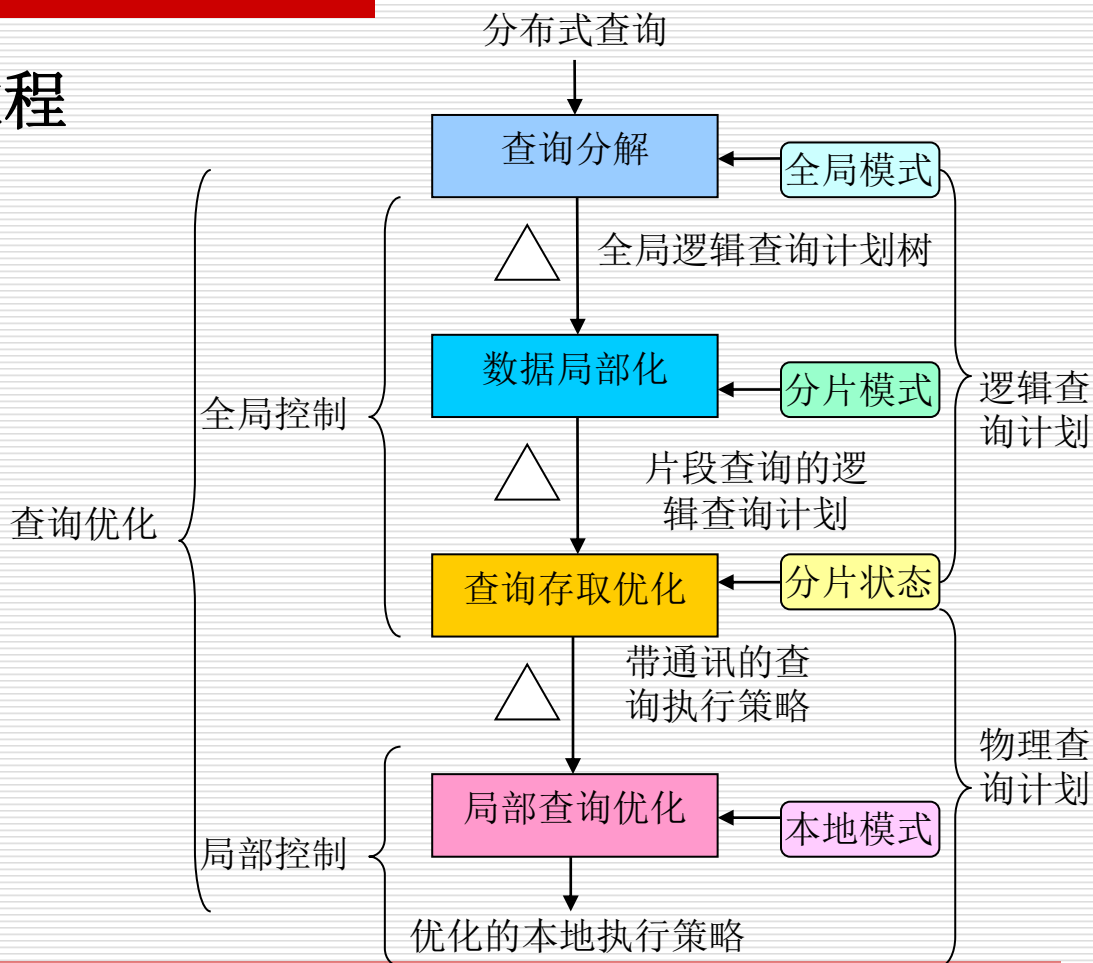
□ 复制的片段

最小化通信代价。

□ 使用半连接

查询处理器

□ 分布式查询处理过程 (4个层次)



查询处理器

□ 查询分解（Query Decomposition）

基于全局概念模式将演算查询分解为代数查询。

Step1 – 演算规范化

Step2 – 语义分析，去掉不正确的查询。

Step3 – 简化，去除冗余的部分

Step4 – 将演算查询转化为优化的代数查询。

□ 数据局部化（Data Localization）

分布查询映射为片段查询，简化、重组为优化的查询。

查询处理器

□ 查询存取优化 (Global Query Optimization)

- 找接近于最优的执行策略;
- 找片段查询中最佳的操作顺序, 包括通信操作;
- 需要实时定义代价函数。

□ 局部查询优化 (Local Query Optimization)

集中的系统算法.

- **INGRES** – 动态优化 (dynamic optimization)
- **System R** – 基于穷举法的静态优化 (static optimization based on exhaustive search)

查询分解（Decomposition）

□ 查询分解（QUERY DECOMPOSITION）

输入：基于全局关系的演算查询

■ 规范化（Normalization）

将查询转换为规范化形式（**AND**形式或**OR**形式）

■ 分析（Analysis）

检查不正确的查询

得到关系演算子集

■ 约简（Simplification）

删除冗余谓词

■ 查询重写（Restructuring）

→ 将演算查询转换为关系代数查询

→ 可能得到多种转换结果

→ 使用转换规则

查询分解 (Decomposition)

■ 规范化

对于SQL语句，对Where子句中的谓词表示进行规范化。

✓ 词法、语法分析

- 有效性检查
- 属性、关系检查
- 限定条件的类型检查

✓ 转换为规范化表示形式

- 合取范式 (**Conjunctive normal form**)
$$(p_{11} \vee p_{12} \vee \dots \vee p_{1n}) \wedge \dots \wedge (p_{m1} \vee p_{m2} \vee \dots \vee p_{mn})$$
- 析取范式 (**Disjunctive normal form**)
$$(p_{11} \wedge p_{12} \wedge \dots \wedge p_{1n}) \vee \dots \vee (p_{m1} \wedge p_{m2} \wedge \dots \wedge p_{mn})$$
- **OR**映射为并操作 (**union**)
- **AND**映射为连接 (**join**) 或选择 (**selection**)

查询分解 (Decomposition)

■ 规范化-转换规则

无量词谓词的转换有如下等价公式:

✓ 交换律:

$$P_1 \wedge P_2 \Leftrightarrow P_2 \wedge P_1$$

$$P_1 \vee P_2 \Leftrightarrow P_2 \vee P_1$$

✓ 分配律:

$$P_1 \vee (P_2 \wedge P_3) \Leftrightarrow (P_1 \vee P_2) \wedge (P_1 \vee P_3)$$

$$P_1 \wedge (P_2 \vee P_3) \Leftrightarrow (P_1 \wedge P_2) \vee (P_1 \wedge P_3)$$

✓ 结合律:

$$P_1 \wedge (P_2 \wedge P_3) \Leftrightarrow (P_1 \wedge P_2) \wedge P_3$$

$$P_1 \vee (P_2 \vee P_3) \Leftrightarrow (P_1 \vee P_2) \vee P_3$$

✓ 德.摩根定律:

$$\neg(P_1 \wedge P_2) \Leftrightarrow \neg P_1 \vee \neg P_2$$

$$\neg(P_1 \vee P_2) \Leftrightarrow \neg P_1 \wedge \neg P_2$$

✓ 对合律:

$$\neg(\neg P_1) \Leftrightarrow P_1$$

查询分解 (Decomposition)

■ 规范化

例: SELECT ENAME
FROM E, G
WHERE E.ENO=G.ENO
AND G.JNO="J1"
AND (DUR=12 OR DUR=24)

合取范式

$$E.ENO=G.ENO \wedge G.JNO="J1" \wedge (DUR=12 \vee DUR=24)$$

析取范式

$$(E.ENO=G.ENO \wedge G.JNO="J1" \wedge DUR=12) \vee \\ (E.ENO=G.ENO \wedge G.JNO="J1" \wedge DUR=24)$$

查询分解 (Decomposition)

■ 分析

目的: 拒绝类型不正确或语义不正确的查询

✓ 类型不正确

没定义的关系、属性, 错误的类型映射 等

例如:

```
SELECT E#           !undefined attribute
```

```
FROM E
```

```
WHERE ENAME>200 !type mismatching
```

查询分解 (Decomposition)

■ 分析

✓ 语义不正确

- 查询中存在对查询结果无意义的部分
- 实际上, 不可能确定一般查询的语义正确性。但对于不包含 $\vee$ 和 $\neg$ 操作的查询是可以判断的。
- 检查语义不正确的方法

查询图 (query graph)

连接图 (join graph) :

只考虑连接的子图称为连接图 (join graph)

查询分解（Decomposition）

■ 分析

✓ 语义不正确

查询图定义如下：

- 一个点表示结果关系，其它点表示操作关系

- 边分为两类

若边的两点都不是结果，则该类边表示连接(join)

若边的一个点为结果点，则该边为投影(projection)

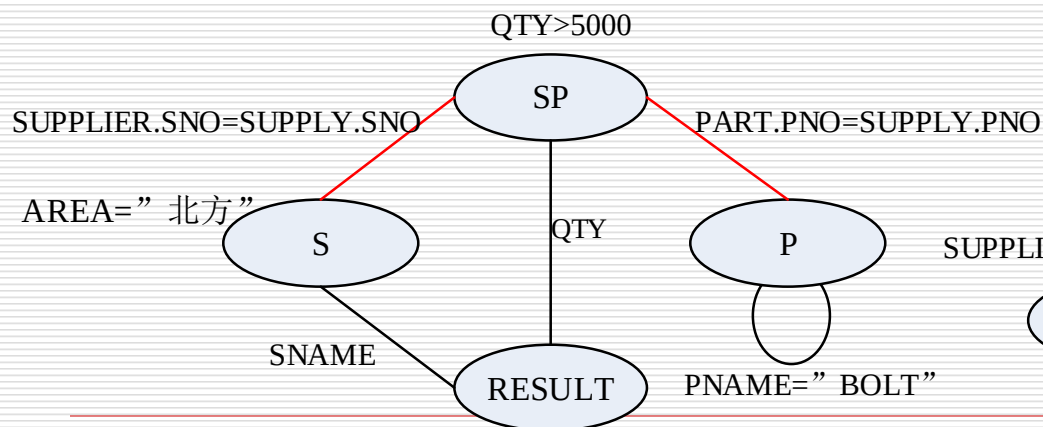
- 点和边被标注为选择谓词（predicates for selection）、投影（projection）或连接（join）

查询分解 (Decomposition)

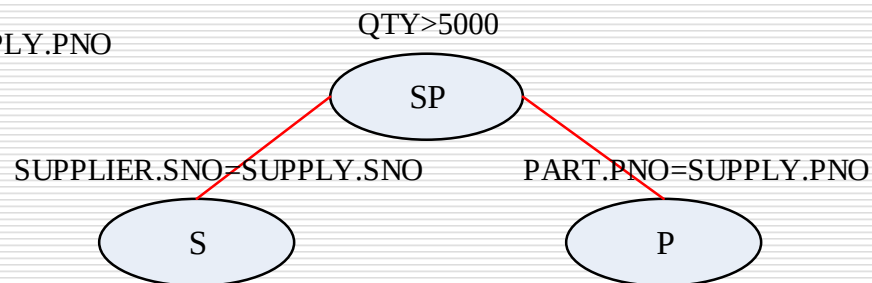
■ 分析-举例

```

SELECT      SNAME
FROM        SUPPLIER S, SUPPLY SP, PART P
WHERE       S.SNO=SP.SNO
AND         SP.PNO=P.PNO
AND         P.PNAME="BOLT"
AND         S.AREA="北方"
AND         SP.QTY>5000;
    
```



(a) 查询图

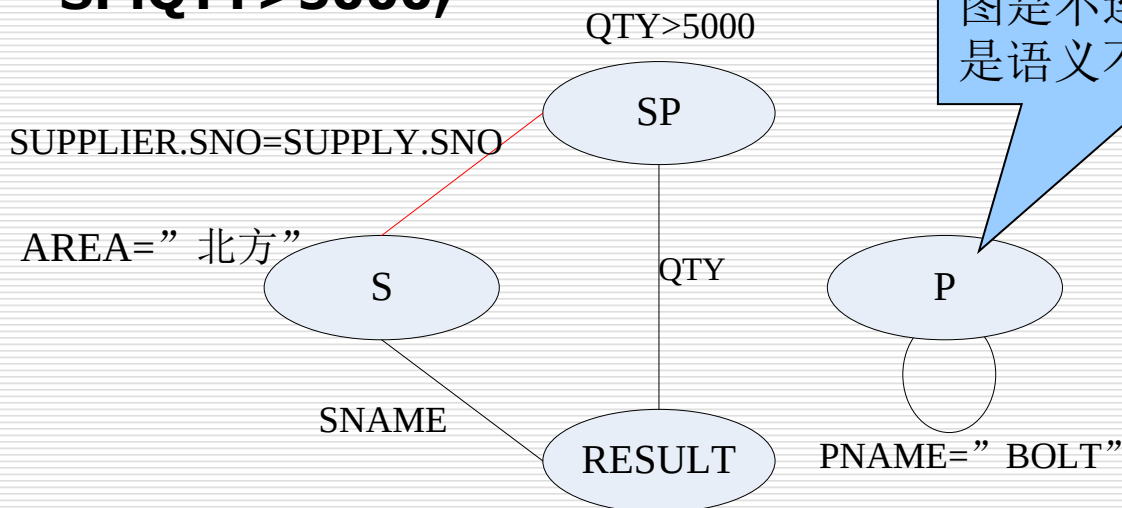


(a) 连接图

查询分解（Decomposition）

■ 分析-举例

```
SELECT      SNAME
FROM        SUPPLIER S, SUPPLY SP, PART P
WHERE       S.SNO=SP.SNO
AND         P.PNAME="BOLT"
AND         S.AREA="北方"
AND         SP.QTY>5000;
```



当一个或者更多的子图与包含结果关系的图是不连通时，认为是语义不正确的

查询分解 (Decomposition)

■ 约简

约简规则:

$$P \wedge P \Leftrightarrow P$$

$$P \vee P \Leftrightarrow P$$

应用转换规则、幂等规则消除冗余

$$P \wedge \text{ture} \Leftrightarrow P$$

$$P \vee \text{false} \Leftrightarrow P$$

$$P \wedge \text{false} \Leftrightarrow \text{false}$$

$$P \vee \text{ture} \Leftrightarrow \text{ture}$$

$$P \wedge \neg P \Leftrightarrow \text{false}$$

$$P \vee \neg P \Leftrightarrow \text{ture}$$

$$P_1 \wedge (P_1 \vee P_2) \Leftrightarrow P_1$$

$$P_1 \vee (P_1 \wedge P_2) \Leftrightarrow P_1$$

查询分解 (Decomposition)

约简-例如

```
SELECT      PNAME
FROM        PART
WHERE       (NOT (PNAME="BOLT")
AND         (PNAME=" SCREW"
OR          PNAME="BOLT")
AND        NOT (PNAME=" SCREW"))
OR          PNAME=" NUT"
```



```
SELECT PNAME
FROM   PART
WHERE  PNAME="NUT"
```

设 P_1 表示 $PNAME = \text{"BOLT"}$, P_2 表示 $PNAME = \text{"SCREW"}$, P_3 表示 $PNAME = \text{"NUT"}$, 则查询可以表示为:

$$(\neg P_1 \wedge (P_1 \vee P_2) \wedge \neg P_2) \vee P_3 \iff (\neg P_1 \wedge ((P_1 \wedge \neg P_2) \vee (P_2 \wedge \neg P_2))) \vee P_3$$

$$\iff (\neg P_1 \wedge P_1 \wedge \neg P_2) \vee (\neg P_1 \wedge P_2 \wedge \neg P_2) \vee P_3$$

$$\iff (false \wedge \neg P_2) \vee (\neg P_1 \wedge false) \vee P_3 \iff false \vee false \vee P_3 \iff P_3$$

查询分解 (Decomposition)

■ 查询重写

- ✓ 直接将关系演算转换为关系代数;
- ✓ 重写关系代数查询以提高性能。

■ 通常使用查询树来表示关系代数查询。

✓ 查询树定义为:

- 根节点表示查询结果
- 叶子节点表示关系;
- 非叶子节点表示中间结果关系
- 叶子到根的边表示序列操作。

查询分解（Decomposition）

■ 查询重写

✓ SQL查询转换为查询树：

- 叶子来自于**FROM**子句；
- 根节点是结果关系，由所需要属性的投影操作生成，包含在**SELECT**子句中；
- **WHERE**子句中的条件被转换成从叶子节点到根节点的关系操作序列，操作序列由操作符和谓词出现的顺序直接生成。

查询分解 (Decomposition)

■ 查询重写

查询重写是将用户请求构成的查询树进行等价变换

规则1: 连接、笛卡尔积的交换律

$$R \times S \Leftrightarrow S \times R, R \bowtie S \Leftrightarrow S \bowtie R$$

规则2: 连接、笛卡尔积的结合律

$$(R \times S) \times T \Leftrightarrow R \times (S \times T), (R \bowtie S) \bowtie T \Leftrightarrow R \bowtie (S \bowtie T)$$

规则3: 投影的串接定律

$$\pi_{A1, A2, \dots, An}(\pi_{B1, B2, \dots, Bn}(E)) \Leftrightarrow \pi_{A1, A2, \dots, An}(E)$$

规则4: 选择的串接定律

$$\sigma_{P1}(\sigma_{P2}(R)) \Leftrightarrow \sigma_{P1 \wedge P2}(R)$$

规则5: 选择和投影的交换律

$$\sigma_P(\pi_{A1, A2, \dots, An}(R)) \Leftrightarrow \pi_{A1, A2, \dots, An}(\sigma_P(R))$$

查询分解 (Decomposition)

■ 查询重写

规则6: 选择与笛卡尔积的分配律

$$\sigma_p(\mathbf{R} \times \mathbf{S}) \equiv \sigma_p(\mathbf{R}) \times \mathbf{S}$$

规则7: 选择与并的分配律

$$\sigma_p(\mathbf{R} \cup \mathbf{T}) \Leftrightarrow \sigma_p(\mathbf{R}) \cup \sigma_p(\mathbf{T})$$

规则8: 选择与差的分配律

$$\sigma_p(\mathbf{R} - \mathbf{T}) \Leftrightarrow \sigma_p(\mathbf{R}) - \sigma_p(\mathbf{T})$$

规则9: 投影与笛卡尔积的分配律

$$\pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_n}(\mathbf{R} \times \mathbf{S}) \equiv \pi_{A_1, A_2, \dots, A_n}(\mathbf{R}) \times \pi_{B_1, B_2, \dots, B_n}(\mathbf{S})$$

规则10: 投影与并的分配律

$$\pi_{A_1, A_2, \dots, A_n}(\mathbf{R} \cup \mathbf{F}) \equiv \pi_{A_1, A_2, \dots, A_n}(\mathbf{R}) \cup \pi_{A_1, A_2, \dots, A_n}(\mathbf{F})$$

查询分解 (Decomposition)

■ 查询重写

- ✓ 应用等价变换规则，一棵查询树可等价转换为多棵查询树，需要能生成最优查询树的等价变换。
- ✓ 关系查询优化的基本思想是先做能使中间结果变小的操作，尽量减少查询执行代价。

运算类型	执行代价
σ 、 Π (不消重复项)	$O(n)$
Π (消重复项)、GROUP	$O(n \log n)$
$\times$	$O(n^2)$
∞ 、 $\cup$ 、 $\cap$ 、 $-$ 、 ∞ 、 $\div$	$O(n \log n)$

变换的通用准则为：

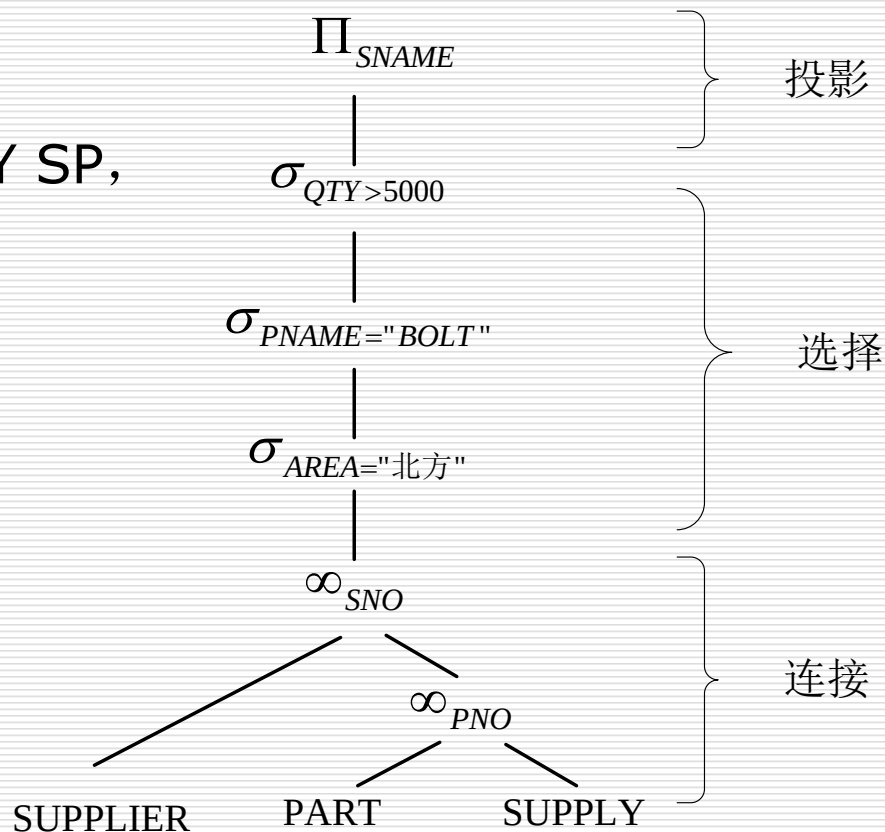
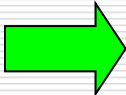
准则1 尽可能将一元运算移到查询树的底部（树叶部分），优先执行一元运算。

准则2 利用一元运算重复律，缩减每一关系，减少关系尺寸，降低网络传输量和I/O大小。

查询分解 (Decomposition)

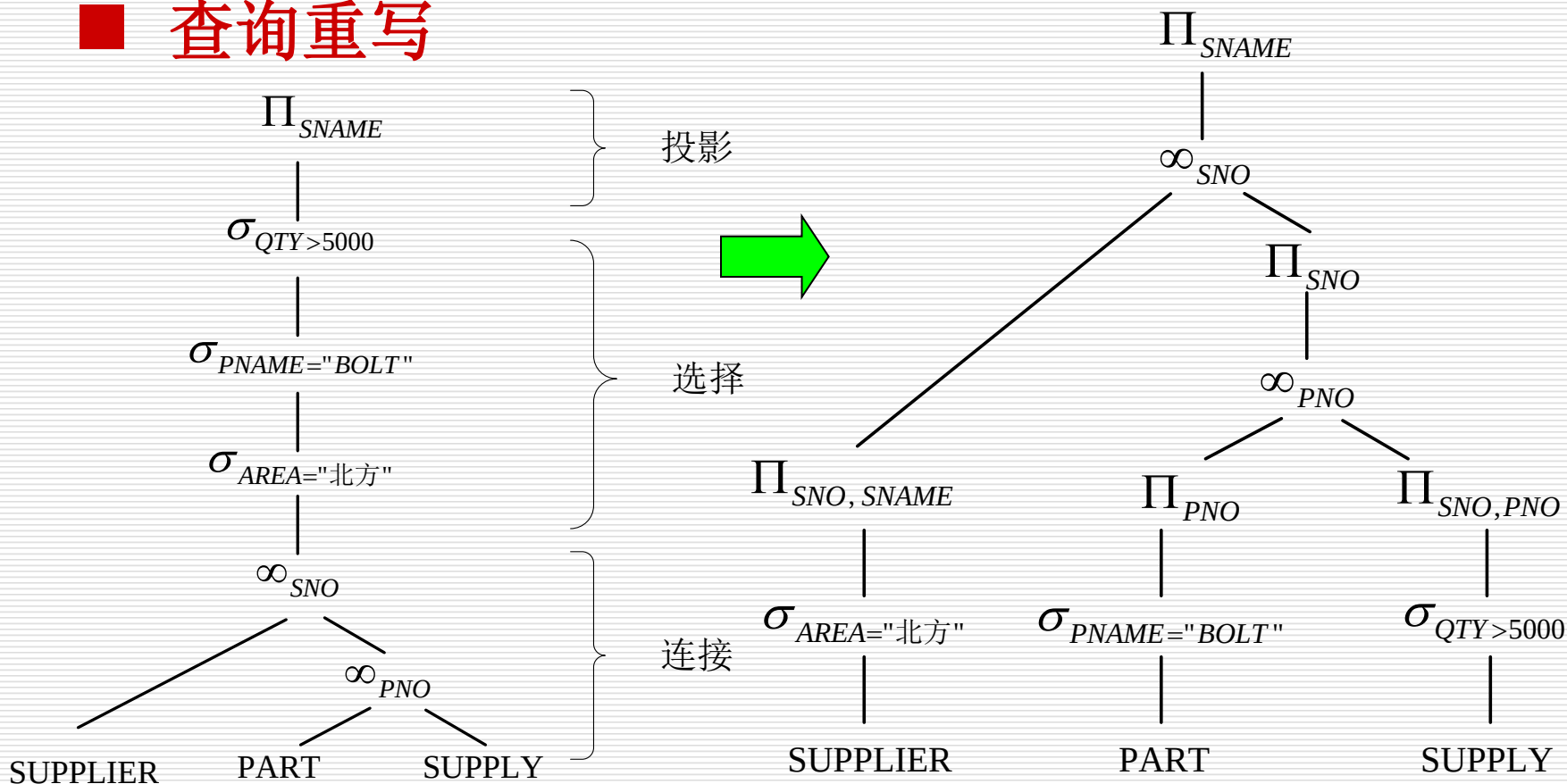
■ 查询重写

```
SELECT SNAME  
FROM SUPPLIER S, SUPPLY SP,  
PART P  
WHERE S.SNO=SP.SNO  
AND SP.PNO=P.PNO  
AND P.PNAME="BOLT"  
AND S.AREA="北方"  
AND SP.QTY>5000;
```



查询分解 (Decomposition)

■ 查询重写



查询分解 (Decomposition)

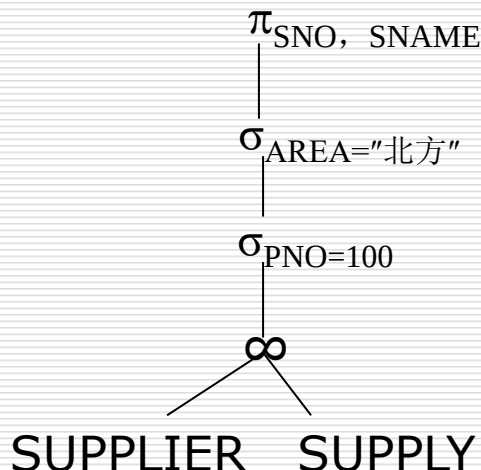
□ 举例

SUPPLIER{SNO, SNAME, AREA}
SUPPLY{SNO, PNO, QTY}

等价的关系表达式:

Q1: $\pi_{SNO, SNAME}(\sigma_{AREA="北方"}(\sigma_{PNO=100}(SUPPLIER \bowtie SUPPLY)))$

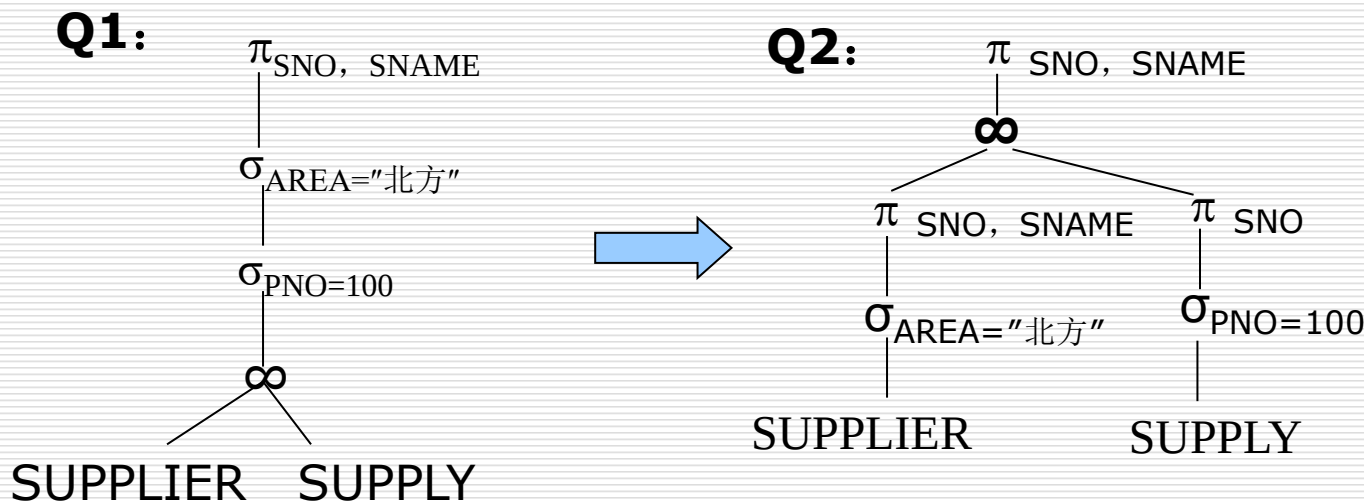
查询树:



查询分解 (Decomposition)

全局优化

基于**查询重写**准则（在查询树**Q1**的基础上进行全局优化）
根据分配律，将一元运算向下移。得到全局优化后的查询树：



数据局部化

□ **查询的处理过程**是从全局关系到片段关系，最后再到实际操作的副本关系。

□ **数据本地化**将介绍**全局查询**到**片段查询**的变换。即利用全局关系与其片段关系的等价变换，将分布查询中的全局关系替换为对片段关系的查询，变换后的查询称为**片段查询**。对应于片段查询的查询树，称为**片段查询树**。

数据局部化

□ 分布查询与片段查询的等价关系

- 对于全局关系R的水平分片 $R_1, R_2, \dots, R_n$ ，表示为：

$$R = R_1 \cup R_2 \cup \dots \cup R_n$$

- 对于全局关系R的垂直分片 $R_1, R_2, \dots, R_n$ ，表示为：

$$R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$$

□ 片段查询树的生成步骤

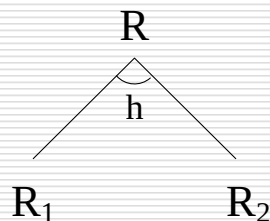
- 将分片树的h（水平）节点转换为查询树的 $\cup$ （并集）节点。
- 将分片树的v（垂直）节点转换为查询树的 $\bowtie$ （联接）节点。
- 用替换后的分片树代替全局查询树中的全局关系，得到片段查询树。

数据局部化

□ 片段查询树的生成步骤

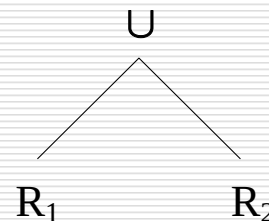
具体如下：

水平h节点分片树

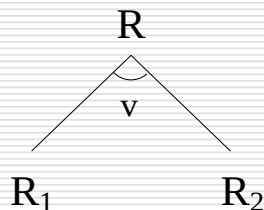


$\implies$

转换后的U节点

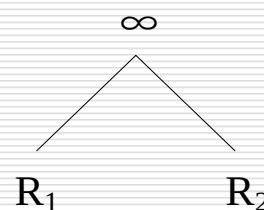


垂直v节点分片树



$\implies$

转换后的 ∞ 节点



数据局部化

□ 例： 假设SUPPLIER和SUPPLY的分片如下：

■ SUPPLIER水平分片为 S_1 和 S_2 ，具体如下：

$$S_1 = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLIER})$$

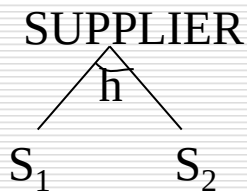
$$S_2 = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLIER})$$

■ SUPPLY水平分片为 SUPPLY_1 和 SUPPLY_2 ，具体如下：

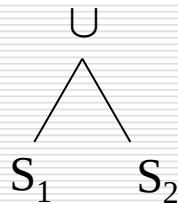
$$\text{SUPPLY}_1 = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLY})$$

$$\text{SUPPLY}_2 = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLY})$$

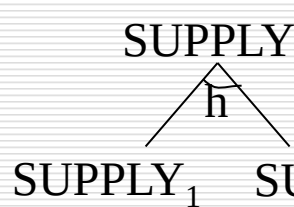
SUPPLIER的分片树为：



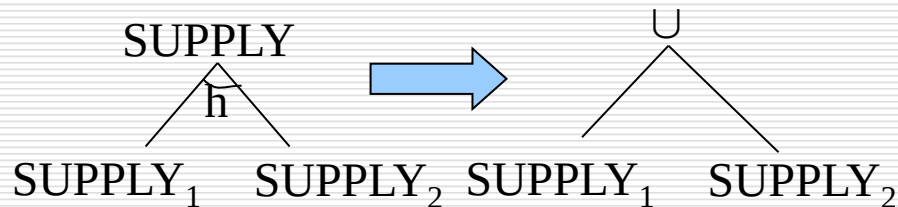
转换后的 $\cup$ 节点：



SUPPLY的分片树为：



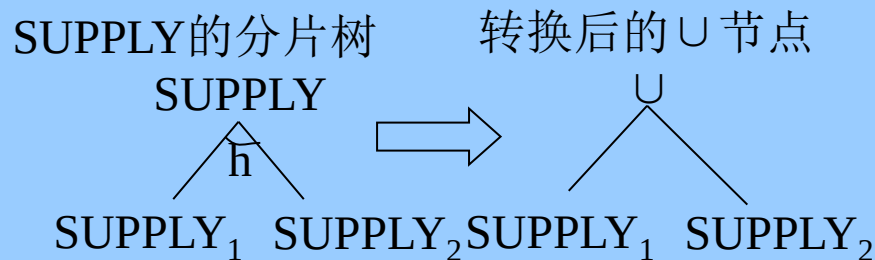
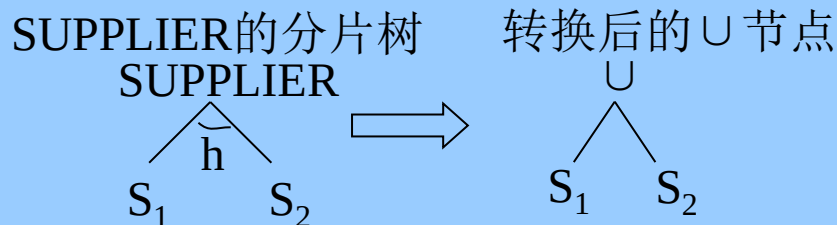
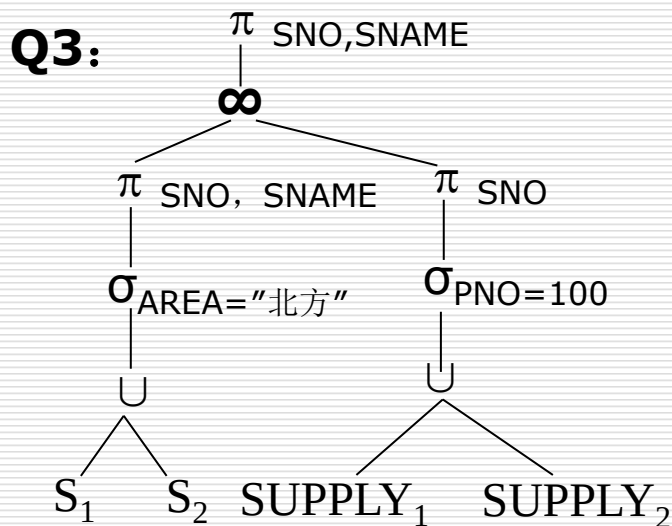
转换后的 $\cup$ 节点：



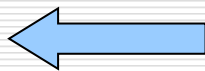
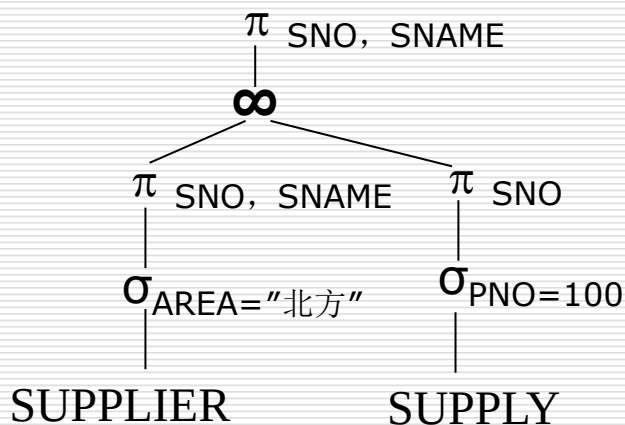
数据局部化

■ 转换为片段查询树Q3

在Q2基础上，用SUPPLIER分片树替换查询树Q2的全局关系SUPPLIER，用SUPPLY分片树替换查询树Q2的全局关系SUPPLY，即得到转换后的片段查询树为Q3。



Q2:



片段查询的优化

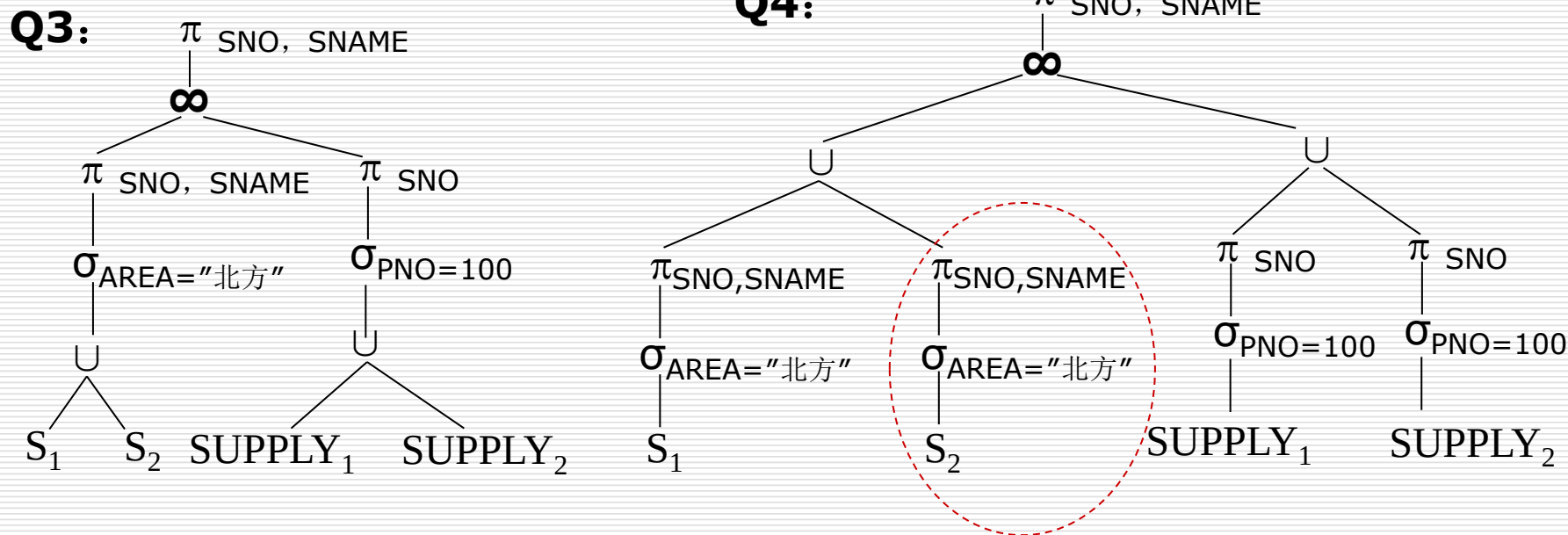
□ 片段查询优化规则

- 准则1:** 对于一元运算，根据一元运算的重复律，将叶子节点之前的选择运算作用于片段，如果不满足片段的限定条件，则置为空关系。
- 准则2:** 对于联接运算的树，若联接条件不满足，则将其置为空关系。
- 准则3:** 在查询树中，将联接运算 ($\Join$) 下移到并运算 ($\cup$) 之前执行。
- 准则4:** 消去不影响查询运算的垂直片段。

片段查询的优化

■ 接上例，以片段查询树**Q3**为基础，

(1) 根据片段查询优化准则1，按限定条件化简，得到**Q4**。



片段查询的优化

(2) 根据

$S_1 = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLIER})$

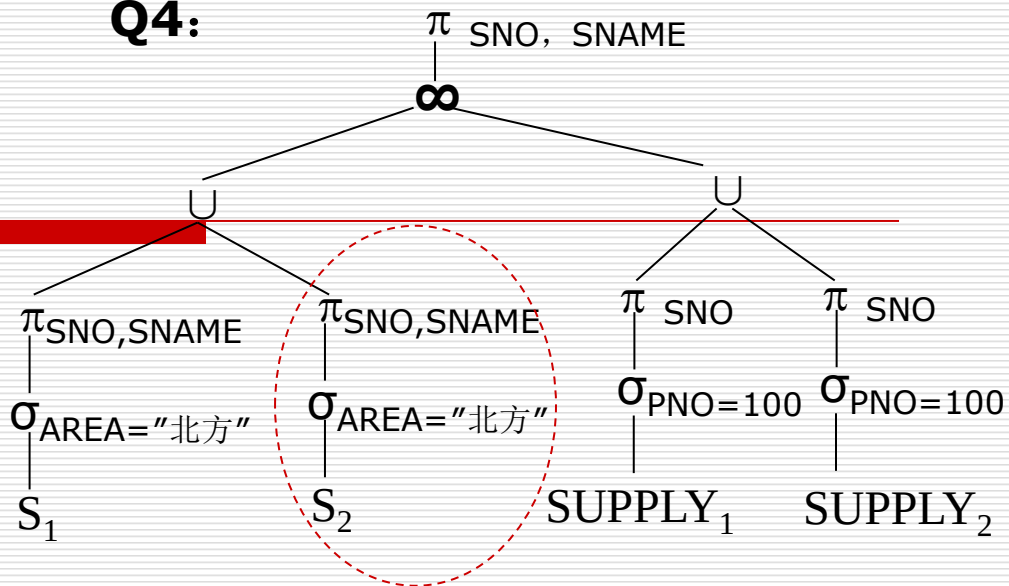
$S_2 = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLIER})$

$\text{SUPPLY}_1 = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLY})$

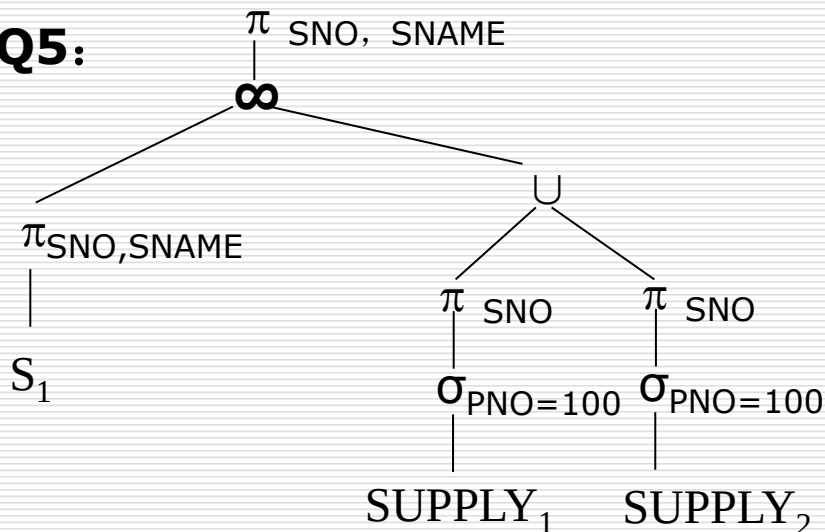
$\text{SUPPLY}_2 = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLY})$

所以，**Q4**中的虚线部分为
空关系，可将其去掉，得到
Q5查询树。

Q4:

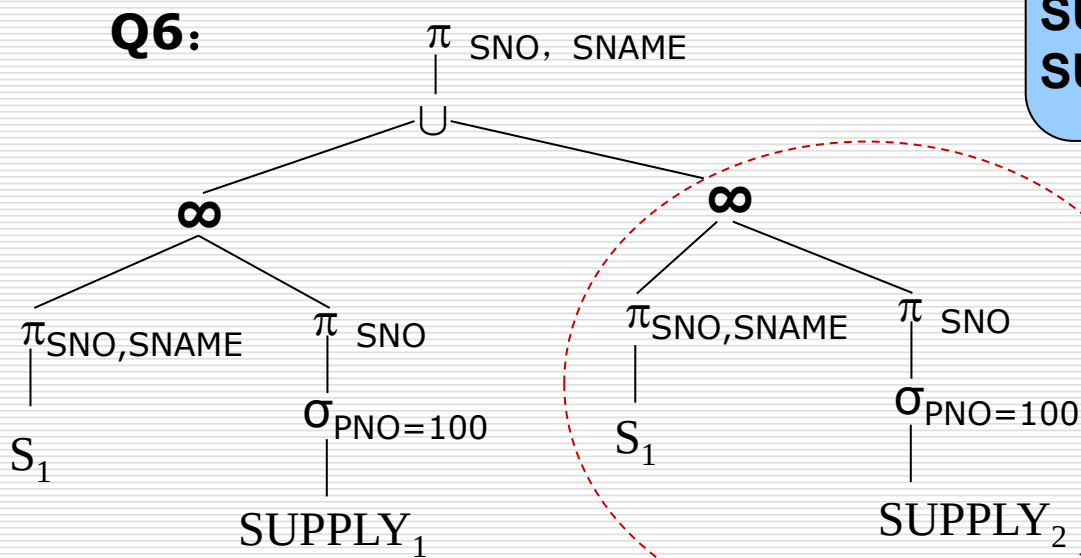


Q5:



片段查询的优化

(3) 根据准则3, 将联接运算($\Join$)下移到并运算($\cup$)之前, 得到Q6。

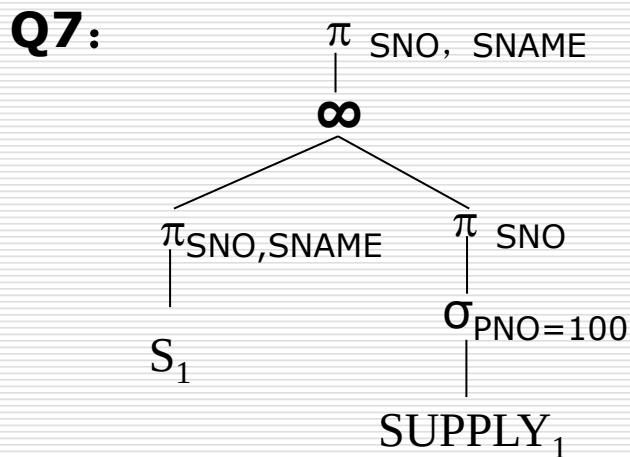


$S1 = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLIER})$
 $S2 = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLIER})$
 $\text{SUPPLY1} = \sigma_{\text{AREA}=\text{"北方"}}(\text{SUPPLY})$
 $\text{SUPPLY2} = \sigma_{\text{AREA}=\text{"南方"}}(\text{SUPPLY})$

则应将其置为
空关系

片段查询的优化

得到Q7:



Q7为化简后的最终的优化查询。

片段查询的优化

□ 例

■ 假设：一雇员关系 EMP {ENO, ENAME, BIRTH, SALARY, DNO}

✓ 其分片为：

$$E1 = \Pi_{\text{ENO, ENAME, BIRTH}} (\text{EMP})$$

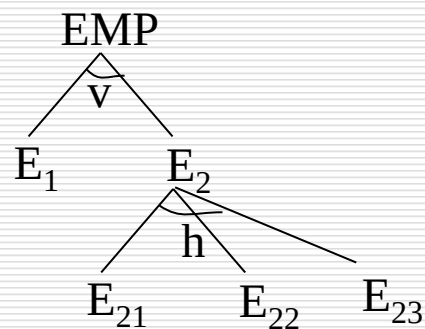
$$E2 = \Pi_{\text{ENO, SALARY, DNO}} (\text{EMP})$$

$$E21 = \sigma_{\text{Dno}=201} (E2)$$

$$E22 = \sigma_{\text{Dno}=202} (E2)$$

$$E23 = \sigma_{\text{Dno} \neq 201 \text{ AND } \text{Dno} \neq 202} (E2)$$

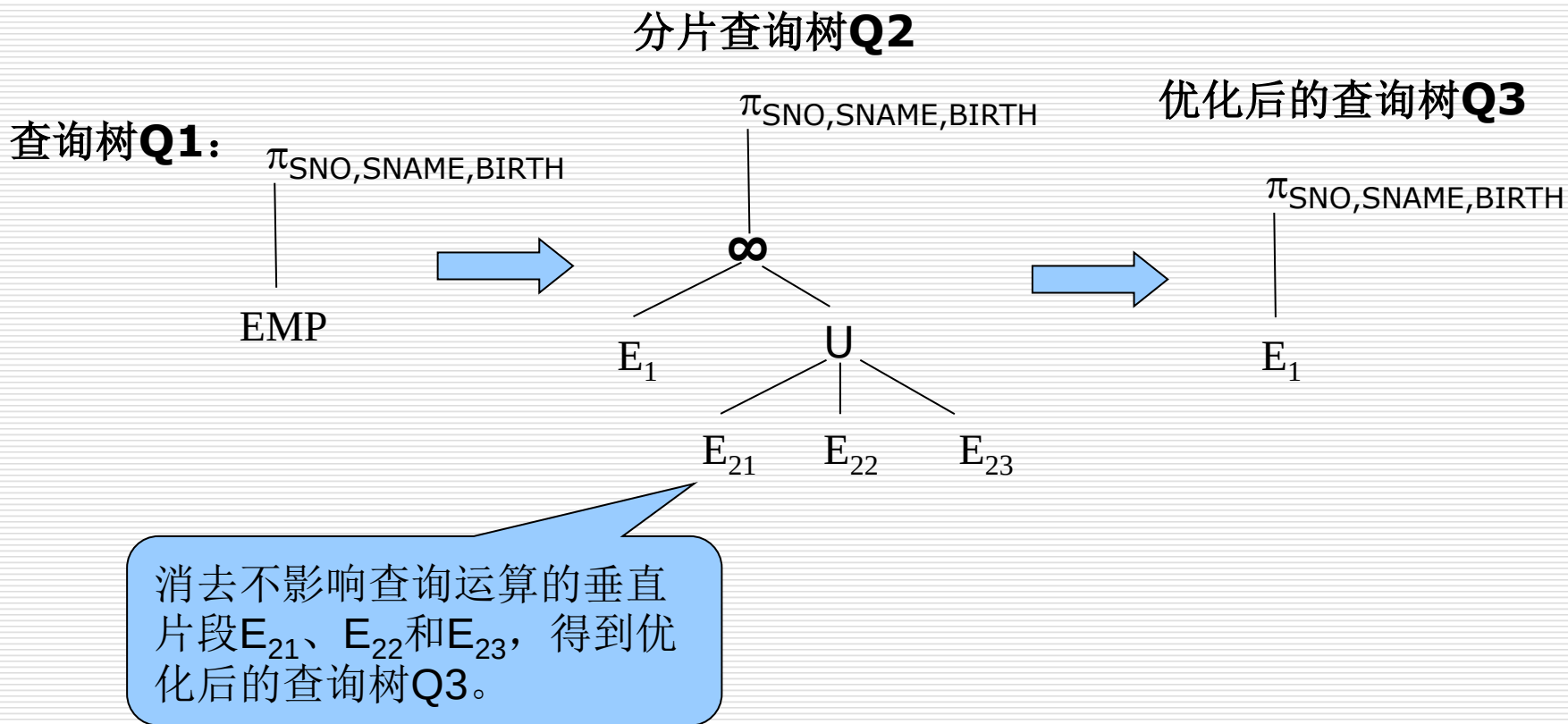
✓ 分片树：



要求：查询SQL：SELECT ENO, ENAME, BIRTH FROM EMP

片段查询的优化

□ 例



大数据数据库的查询处理及优化

□ 大数据数据库查询处理方法

■ 键值型数据库的查询处理

(key, Value): 基于key查询其对应的值。

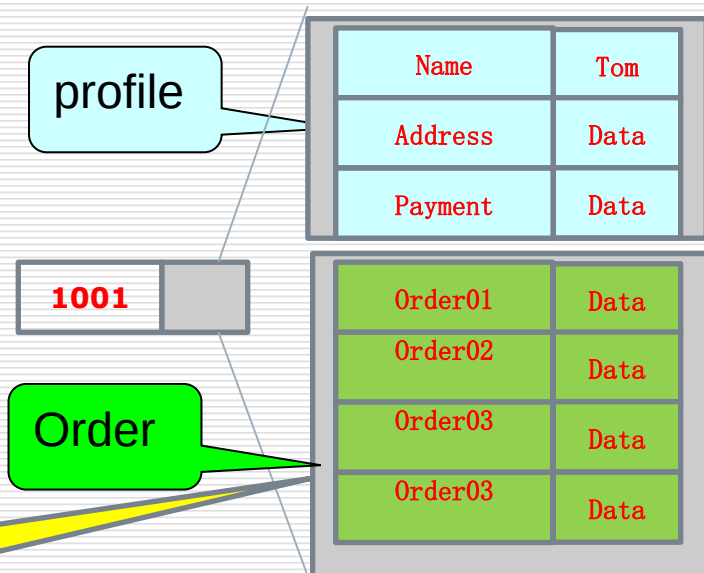
■ 列存储数据库查询处理

以列为核心存储，具有**相同列属性**的数据集存储在一个页面或数据块中，快速响应**以列为主**的查询。

例如：以列族结构表示的客户信息，将每个用户的信息存储在一起。

■ 文档数据库查询处理

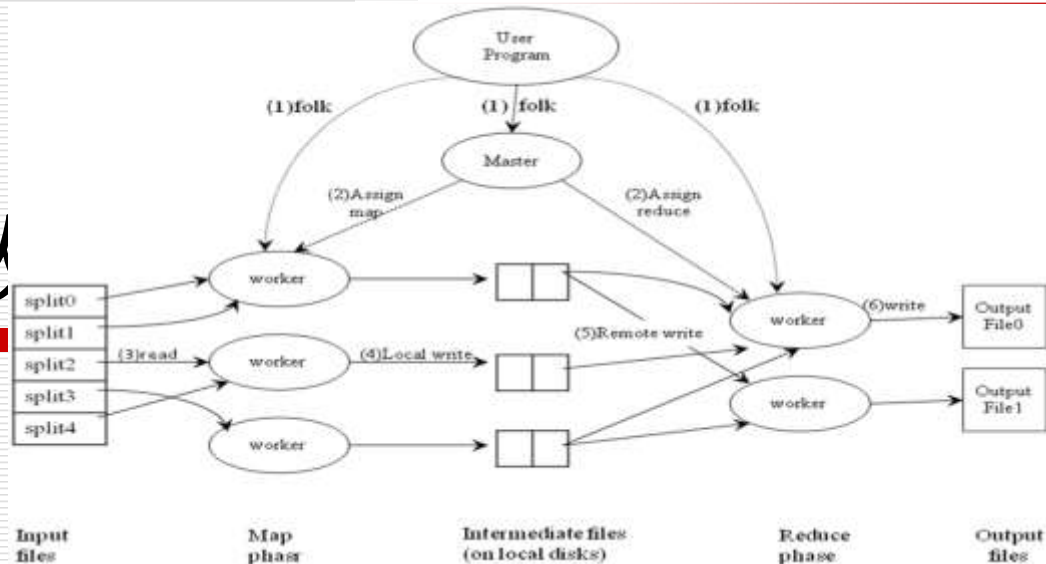
文档数据库的数据格式主要包括XML、JSON、BSON等，典型通过视图查询



文档1: {"firstname": "Tom",
"likes":["Biking","Photography"],
"lastcity":"Boston",
"lastVisited": }

大数据的查询

□ MapReduce框架处理流程



- 1) 把输入文件**划分为M份**（M由用户定义），通常每份16MB到64MB，然后启动多个程序副本到集群内其它机器上。
- 2) 用户程序的副本中的一个称为master，其余称为worker，**worker由master分配任务**，包括M个**Map任务**和R个**Reduce任务**。
- 3) 被分配了Map作业的worker，读取对应输入分片的数据，**Map从输入数据中抽取出键值对**。
- 4) 缓存的**中间键值对**会被定期**写入本地磁盘**，而且通过分区函数被分为R个区，master负责将信息转发给Reduce。
- 5) Reduce从Map的本地磁盘**读取**它负责的缓存的**中间键值**，对它们进行**排序**，使得相同键的键值对聚集在一起。
- 6) Reduce遍历排序后的中间键值对，**reduce函数产生的输出**添加到该分区的输出文件中。
- 7) 当所有的Map和Reduce**作业都完成了**，master唤醒用户程序，此时，MapReduce函数调用**返回到用户程序**代码。



第四章 结束

